Inclusify — Development Guide
Group G07

Shahaf Wieder
Barak Sharon
Rasha Daher
Lama Zarka
Adan Daxa

Version 1.0
August 2026

Table of Contents

1. Introduction

Inclusify is an NLP-based web platform developed for the Achva LGBT organization. The system analyzes academic texts in Hebrew and English and identifies language that may be LGBTQphobic, outdated, biased, potentially offensive, or pathologizing. It provides severity-graded findings, explanations, inclusive alternatives, and downloadable reports.

This Development Guide is intended for future developers who may maintain, modify, or extend the Inclusify project. Its target audience includes a hired developer, a student continuing the project, or an Achva contractor. The reader is expected to have solid Python and TypeScript skills, but no previous knowledge of the Inclusify codebase.

The purpose of this guide is to help a new developer understand the repository structure, run the system in a development environment, make changes safely across the frontend, backend, database, and machine-learning layers, test those changes, and prepare them for production deployment.

The reader should be familiar with Python, FastAPI, TypeScript, Next.js, PostgreSQL, Docker, Git, and GitHub. Basic familiarity with large language models, LoRA fine-tuning, and REST APIs is also recommended.

This guide complements the other project documents. The **User Guide** explains how users interact with the platform, while the **Technical Guide** explains how to install, configure, and operate the system. This Development Guide focuses on how to change and extend the project. Installation and general operational procedures should therefore be taken from the Technical Guide, while this document covers only development-specific topics such as hot reload, local development workflows, testing, architectural decisions, and safe modification procedures.

2. Repository Tour

The Inclusify repository is organized as a monorepo containing the frontend, backend, database, machine-learning pipeline, infrastructure configuration, documentation, and development utilities. This section provides a high-level map of the repository so that a new developer can quickly identify where each part of the system is implemented.

Directory	Purpose
`frontend/`	Contains the Next.js 16 frontend application, including locale-based routing for Hebrew and English, UI components, translations, authentication state, and the centralized API client.
`backend/app/`	Contains the FastAPI backend, including application startup, API routers, authentication, document ingestion, text analysis, administration, user profiles, contact forms, and feedback handling.
`backend/app/db/`	Contains the asynchronous PostgreSQL connection, FastAPI database dependencies, and repository layer used for all persistence operations.
`db/`	Contains the canonical PostgreSQL schema in `schema.sql` and the initial database content in `seed.sql`.
`ml/`	Contains the model-training pipeline, curated and synthetic data tools, frozen evaluation sets, LoRA adapters, retraining outputs, and the authoritative retraining runbook.
`data/`	Contains the original, augmented, bilingual, and expert-reviewed datasets used during training, evaluation, and data curation.
`infra/docker/`	Contains the Dockerfiles used by Railway to build and deploy the frontend and backend services.
`infra/modal/`	Contains the Modal application that serves the fine-tuned model through vLLM on a serverless GPU.
`scripts/`	Contains project-level utilities such as database checks and machine-learning environment setup scripts.
`docs/`	Contains the delivered project documents (cover letter, user, technical, and development guides), the project poster, screenshots, and media assets.
`.github/workflows/`	Contains the GitHub Actions workflows that run automated quality checks and integration tasks on pushes and pull requests.

```
Inclusify/
├── .github/
│   └── workflows/
├── backend/
│   └── app/
│       └── db/
├── data/
├── db/
├── docs/
├── frontend/
├── infra/
│   ├── docker/
│   └── modal/
├── ml/
│   ├── adapters/
│   ├── data_curated/
│   ├── data_synthesis/
│   ├── eval_sets/
│   ├── retrain_run/
│   ├── scripts/
│   └── training/
└── scripts/
```

Figure 1. High-level structure of the Inclusify repository.

3. Development Environment

Inclusify supports two local development workflows: running the complete stack with Docker Compose, or running the frontend and backend directly on the host while keeping PostgreSQL, Redis, and MinIO in Docker.

First, clone the repository:

```
git clone https://github.com/achvalgbt/Inclusify.git

cd Inclusify
```

Then copy the environment template:

```
cp .env.example .env
```

The values in `.env.example` are configured for Docker Compose. When services run natively, Docker hostnames such as `postgres` and `minio` must be replaced with `localhost`.

3.1 Docker Compose

The simplest development setup is:

```
docker compose --profile dev up
```

This starts the frontend, backend, PostgreSQL, Redis, and MinIO with hot reload enabled. The frontend is available at `http://localhost:3100`, and the backend at `http://localhost:8000`.

3.2 Native Development

The supporting services can continue running in Docker:

```
docker compose --profile dev up postgres redis minio minio-init
```

The backend and frontend can then be started separately:

```
cd backend

uvicorn app.main:app --reload --port 8000

cd frontend

npm run dev
```

In this setup, the frontend uses port `3000`, and the environment variables must point to the host ports of PostgreSQL, Redis, and MinIO.

3.3 Model Access

Most parts of the application can run without the model service. However, actual text analysis requires a reachable vLLM endpoint configured through `VLLM_URL`.

3.4 Tests

Backend development dependencies are installed from `backend/requirements-dev.txt`. Run the backend tests using the project virtual environment:

```
.venv/bin/python -m pytest
```

Backend tests and tooling require Python 3.12 or later. The system Python 3.9 should not be used because it may fail during imports.

Frontend development commands are:

```
npm run dev

npm run lint

npm test
```

Detailed testing information appears in Section 9.

4. Backend Architecture

The Inclusify backend is an asynchronous FastAPI application organized into shared infrastructure, authentication, database access, and feature modules.

4.1 Application Entry Point

`backend/app/main.py` creates the FastAPI application, configures CORS and logging, registers the routers, and manages startup and shutdown.

At startup, it creates the asyncpg pool, ensures the storage bucket exists, warms Docling, initializes Redis, and prepares the authentication tables. At shutdown, it closes Redis and the database pool. Logging is intentionally split so `INFO` and `DEBUG` go to stdout, while `WARNING` and higher go to stderr, because Railway determines severity from the output stream.

4.2 Core Services

The `core/` directory contains shared infrastructure:

- `config.py` loads environment variables through `pydantic-settings`.
- `blob_storage.py` provides S3-compatible storage using Cloudflare R2 in production and MinIO in development. Because `boto3` is synchronous, storage operations run in an executor.
- `redis.py` manages refresh-token storage and Redis connection pooling.

4.3 Authentication and RBAC

Authentication is implemented with FastAPI Users.

`users.py` configures the user database adapter and exports the login, registration, and user-management routers. `backend.py` defines the JWT backend and adds the user role to token claims. `manager.py` handles user lifecycle events and password-reset emails, while `oauth.py` implements Google OAuth and account linking by email.

Role checks are defined in `auth/deps.py` through `require_role` and `require_admin`. The valid database roles are `user` and `site_admin`; `org_admin` remains only as a historical code artifact.

4.4 Document Ingestion

The `modules/ingestion/` module handles uploads, text extraction, metadata, language detection, OCR, and chunk creation.

Docling runs in a recycled subprocess created with the `spawn` method, using `ProcessPoolExecutor` with `max_tasks_per_child=4`. This is intentional: running it inside the API process caused memory growth and out-of-memory failures. Parsing is serialized so several heavy conversions do not run at the same time.

4.5 Analysis

The `modules/analysis/` module performs inclusive-language detection.

Despite its name, the `HybridDetector` class name is historical; detection is currently **LLM-only**, with no rule-based pass. It splits or locates chunks, sends them to the model, validates the results, removes duplicates, and maps findings back to the source text.

`sentence_splitter.py` uses `pysbd` to split Hebrew and English text while preserving the character offsets required for highlighting findings in the original document.

`llm_client.py` handles communication with vLLM, including bearer authentication, timeouts, circuit-breaker protection, JSON parsing, severity mapping, and logprob-based confidence. `scoring.py` calculates the score from the proportion of document characters not covered by findings.

The analyze endpoint is rate-limited to 20 calls per hour per caller — per user when signed in, per client IP otherwise — and caps request text at 200,000 characters. Both limits are defined in backend/app/modules/analysis/router.py and exist because every call occupies a paid GPU.

4.6 Supporting Modules

- `admin/` provides analytics, user and role management, model metrics, and feedback administration.
- `profile/` provides profile and analysis-history endpoints.
- `contact/` handles contact-form submissions through Resend.
- `feedback/` stores user feedback on findings.

4.7 Repository Layer

All raw asyncpg SQL is centralized in `backend/app/db/repository.py`; routers should not write SQL directly.

The repository contains functions for documents, analysis runs, findings, suggestions, user history, and administrative queries. New persistence behavior should be added as a repository function and then called from the relevant router or service.

4.8 Adding a Backend Endpoint

1. Create or update a router under the relevant `backend/app/modules/` package.
2. Define request and response models with Pydantic.
3. Add database operations to `backend/app/db/repository.py` when persistence is required.
4. Register the router in `backend/app/main.py` only when creating a new module.
5. Add or update backend tests under `backend/tests/`.

5. The Analysis Pipeline End-to-End

Figure 2 summarizes the complete Inclusify analysis flow, from document upload to the response returned to the frontend.

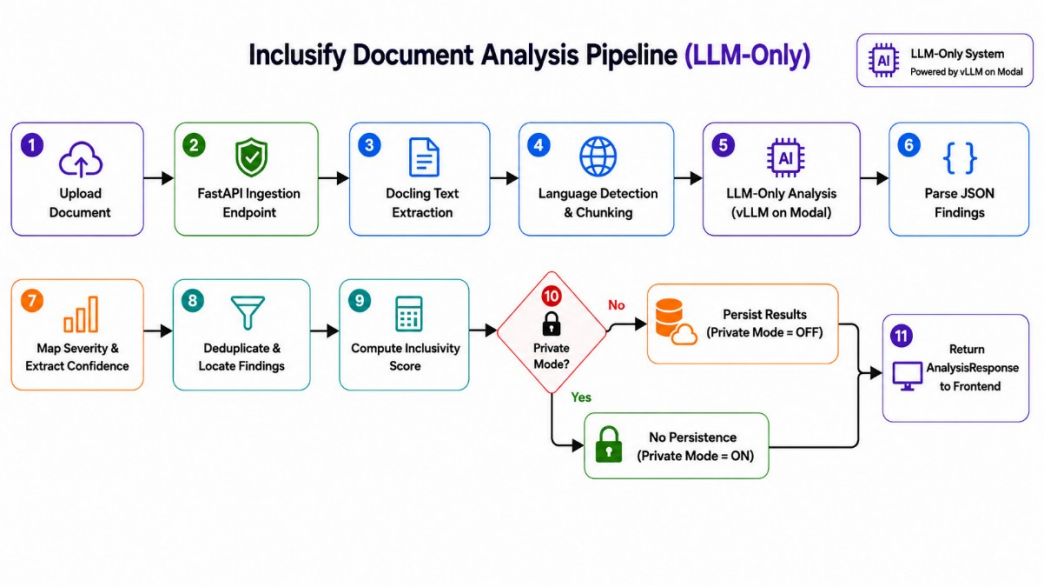

Figure 2. End-to-end Inclusify document analysis pipeline.

1. **Upload Document**
   The user uploads a PDF, DOCX, PPTX, or TXT file through the FastAPI ingestion endpoint.

2. **Ingestion and Validation**
   The endpoint validates the file type and enforces a maximum file size of 50 MB before reading the file contents. Docling then enforces a 50-page cap, so a document is rejected when it exceeds either limit, whichever is reached first.

3. **Docling Text Extraction**
   Docling extracts the document text, metadata, page information, and layout-aware chunks. When required and enabled, scanned PDFs are processed using Hebrew and English OCR.

4. **Language Detection and Chunking**
   The system detects whether the extracted text is primarily Hebrew or English and divides it into chunks for analysis. When Docling chunks are unavailable, pysbd is used for sentence splitting while preserving character offsets.

5. **LLM-Only Analysis**
   Each chunk is sent through vLLM on Modal to the fine-tuned model served under the name `inclusify`. The request includes the production system prompt, which defines the categories, severity levels, output format, and language rules. Detection is LLM-only, with no active rule-based pass.

6. **Parse JSON Findings**
   The model returns JSON findings containing the problematic phrase, category, severity, explanation, and suggested alternative. The backend parses and validates the response.

7. **Map Severity and Extract Confidence**
   Severity values are mapped to the API format, and confidence values are derived from the model log probabilities.

8. **Deduplicate and Locate Findings**
   Duplicate and overlapping findings are removed. The remaining findings are mapped back to exact character positions in the source text.

9. **Compute Inclusivity Score**
   The score is calculated as the percentage of document characters not covered by findings. Overlapping spans are merged so the same character is not counted more than once.

10. **Private Mode and Persistence**
    When Private Mode is enabled, the original file, extracted text, findings, and suggestions are not stored. Only anonymous model-performance metrics may be recorded. When Private Mode is disabled, the original file and extracted text are stored in R2, while the document, analysis run, findings, and suggestions are persisted in PostgreSQL through the repository layer.

11. **Return `AnalysisResponse`**
    The backend returns an `AnalysisResponse` containing the original text, `issues_found`, analysis status, score, flagged-character counts, detected language, and the persisted `run_id` when applicable.

6. Frontend Architecture

The Inclusify frontend is a Next.js 16 application using the App Router. Pages are organized under `frontend/app/[locale]/`, where each folder represents a route such as `analyze`, `history`, `profile`, `glossary`, and `admin`. The `[locale]` segment allows the same page structure to support both Hebrew and English. Shared interface elements are stored under `frontend/components/`, with feature-specific folders for authentication, the dashboard, glossary, landing page, and reusable UI components.

6.1 Internationalization

Internationalization is implemented with `next-intl`. The supported locales are defined in `frontend/i18n/config.ts`:

- `en` — English
- `he` — Hebrew
- English is the default locale.

All user-visible strings must be stored in both:

- `frontend/messages/en.json`
- `frontend/messages/he.json`

The two files use the same nested keys, allowing components to retrieve the appropriate text through `useTranslations`. Hebrew pages are rendered in RTL, while English pages are rendered in LTR. Text should not be hard-coded directly inside components.

6.2 API Layer

API communication is centralized under `frontend/lib/api/`. Components should not call `fetch` directly.

`client.ts` contains the main upload and analysis functions, transforms backend responses into frontend annotations and result cards, handles timeouts and cancellation, and provides `fetchWithAuth` for authenticated requests. A `401` response clears the stored session and redirects the user to the login page.

Additional API files separate feature-specific logic:

- `auth.ts` handles login, registration, password reset, user details, and profile requests.
- `admin.ts` contains authenticated dashboard requests and SWR hooks for analytics, users, model metrics, feedback, and role management.

New API operations should be added to this layer and then called from pages or components.

6.3 Authentication State

Authentication state is managed by `frontend/contexts/AuthContext.tsx`. The context exposes the current user, loading state, login, registration, logout, token access, and profile refresh.

The JWT and its expiry date are stored in `localStorage`. When the application loads, the context validates the saved token and retrieves the current user and profile. Invalid or expired tokens are removed automatically.

6.4 Client-Side Report Export

Analysis reports are generated in the browser by `frontend/lib/exportReport.ts`. The module uses jsPDF to create downloadable reports, including findings, scores, explanations, suggestions, and category summaries.

It supports English and Hebrew output. For Hebrew reports, it loads a Hebrew-compatible font and applies bidirectional text handling so the exported PDF is rendered correctly in RTL.

6.5 Adding a Translated String

To add new user-visible text:

1. Add the same key to both `frontend/messages/en.json` and `frontend/messages/he.json`.
2. Provide the English and Hebrew values.
3. In the component, load the relevant namespace with `useTranslations`.
4. Replace hard-coded text with the translation key.

Example:

```
const t = useTranslations('analyzer');

return <Button>{t('analyzeBtn')}</Button>;
```

Both JSON files must be updated in the same change so neither locale displays a missing key.

6.6 Adding a Page

To add a new localized page:

1. Create a folder under `frontend/app/[locale]/`.
2. Add a `page.tsx` file inside the new folder.
3. Add all user-visible strings to both translation files.
4. Use existing components and the centralized API layer where possible.
5. Add a locale-aware navigation link to the relevant navbar, dashboard, or page.

For example, creating `frontend/app/[locale]/resources/page.tsx` produces the routes `/resources` and `/he/resources`. English is the default locale, so its routes are served without a locale prefix.

This structure keeps routing, translations, API access, authentication, and report generation consistent throughout the frontend.

7. Database

The Inclusify database uses PostgreSQL. The canonical schema is defined in `db/schema.sql`, which is the single source of truth for all database tables, constraints, indexes, and relationships. Initial data is defined in `db/seed.sql`. During the first Docker Compose startup, both files are applied automatically to create and seed the database.

7.1 Changing the Schema Safely

The project does not use a migration framework. Every schema change must therefore follow this procedure:

1. Update `db/schema.sql` so new installations receive the correct schema.
2. Write the equivalent SQL change, usually an `ALTER TABLE` statement.
3. Apply that statement manually to the production database.
4. Document the change in the pull request.

Skipping the first step causes fresh installations to differ from production. Skipping the third step leaves production on an older schema. Both cases create schema drift, which has already occurred in this project.

7.2 Main Tables

The main application tables are:

- `users` — stores accounts, authentication fields, profile information, and roles.
- `documents` — stores document metadata, language, privacy mode, and optional R2 storage references.
- `analysis_runs` — records each analysis, including status, model version, runtime, configuration, and Inclusivity Score.
- `findings` — stores detected issues, their source-text positions, severity, confidence, and explanations.
- `suggestions` — stores replacement text linked to individual findings.
- `feedback` — stores user feedback, including helpful, false-positive, and false-negative reports.
- `model_metrics` — stores privacy-safe vLLM performance information such as call counts, errors, timeouts, and latency.

7.3 Important Constraints

Two constraints are especially important:

- The `users.role` field accepts only `user` and `site_admin`. Other role names that remain in older code or documentation are not valid database roles.
- The Private Mode constraint prevents stored extracted text when `private_mode = TRUE`. In that case, `text_storage_ref` must be `NULL`.

These constraints enforce authorization consistency and the product's privacy promise directly at the database level.

7.4 Database Access

The backend creates an `asyncpg` connection pool during startup. FastAPI routes obtain connections through `get_db`, which releases them automatically and returns `503` when the pool is unavailable or exhausted. All raw SQL should remain in `backend/app/db/repository.py`; routers should not write SQL directly.

SQLAlchemy models are used mainly for FastAPI Users and OAuth integration. They map framework field names such as `id` and `hashed_password` to the canonical database columns `user_id` and `password_hash`, without changing `schema.sql`.

7.5 Currently Unused Tables

The schema still includes `glossary_terms` and `rules`. Seed data inserts an initial guideline source and glossary term, but the current analysis pipeline does not read either table because detection is LLM-only.

8. ML Pipeline and Model Retraining

Inclusify uses `Qwen/Qwen2.5-3B-Instruct` as its base model. The production LoRA adapter is `qwen_r8_d0.2_achva_v2` with rank 8 and dropout 0.2. It is served through vLLM on Modal under the model name `inclusify`.

8.1 Data Assets

The ML-related data is organized as follows:

- `data/` contains the original, augmented, and bilingual synthetic datasets.
- `ml/data_synthesis/` contains the dataset-generation and Hebrew-translation pipeline.
- `ml/data_curated/` contains cleaned, relabeled, and traceable training data.
- `ml/eval_sets/` contains the frozen expert and gold evaluation sets.
- `ml/adapters/` contains the packaged LoRA adapters.

The final retraining dataset contained approximately 14,000 examples, balanced across Hebrew and English. During curation, corrupt rows, duplicates, evaluation-set overlaps, and poor explanations were removed. Use-mention cases were also relabeled so that academic text describing or criticizing problematic views is not automatically treated as a violation.

The evaluation sets are frozen and protected by SHA-256 checksums. They must not be edited or used for training, because changes would invalidate comparisons between model versions.

8.2 Retraining Procedure

`ml/retrain_run/RUN_LOG.md` records the run that produced the current production adapter, and the training and evaluation assets live under ml/training/ and ml/eval_sets/. The procedure is summarized below rather than repeating its commands:

1. Build and freeze the expert and gold evaluation sets.
2. Measure the current production adapter as the baseline.
3. Clean, relabel, deduplicate, and balance the training data.
4. Convert the examples to the same JSON format used in production.
5. Train candidate LoRA adapters on a rented cloud GPU.
6. Evaluate each candidate against the frozen sets using pass/fail criteria.
7. Package the selected adapter with its configuration, metrics, and provenance.

A candidate is accepted only if it reduces use-mention false positives, improves Hebrew macro-F1, avoids a significant English regression, and produces acceptable Hebrew output.

8.3 Adapter Versioning and Promotion

Adapters are stored under `ml/adapters/` and follow the conventions in `ml/adapters/VERSIONING.md`. Old adapters are retained so they remain available for rollback.

To promote a new adapter, update the adapter directory used in `infra/modal/vllm_app.py` and deploy the Modal application:

```
modal deploy infra/modal/vllm_app.py
```

The model continues to be exposed to the backend under the name `inclusify`. Rollback is performed by restoring the previous adapter and redeploying. `ml/scripts/switch_adapter.py` can assist with adapter switching and validation.

8.4 Current Quality Snapshot

As of July 2026, the selected adapter achieved approximately **83.3% accuracy** and **0.706 macro-F1** on the strict expert evaluation set. The use-mention false-positive rate decreased from 1.00 to 0.118. The model was trained on approximately 14,000 examples.

These metrics are based on a relatively small frozen evaluation set and should be treated as directional.

9. Testing

Inclusify includes backend, frontend, integration, accessibility, and load tests. Each layer should be tested with its own tools before a change is merged.

9.1 Backend Tests

Backend tests are stored in `backend/tests/` and use `pytest`. The suite contains approximately 30 files covering authentication and RBAC, administration, analysis, Hebrew processing, ingestion, Docling, storage, scoring, logging, OAuth, password reset, and report persistence.

Run the full backend suite with the project virtual environment:

```
.venv/bin/python -m pytest
```

The project virtual environment must be used because the backend tests and tooling require Python 3.12 or later.

9.2 Backend Test Isolation

`backend/tests/conftest.py` sets an isolated SQLite test database before any application module is imported. This ordering is essential because the authentication engine is created from cached settings during import. Removing this guard may cause tests to bind to the real PostgreSQL configuration and fail against, or potentially affect, the live schema.

Settings are also cached as a singleton when first imported. Tests that modify environment variables after the application or settings module has already been imported may not see the new values. Environment-dependent values should therefore be set before importing application modules.

9.3 vLLM Integration Tests

Tests that require a live vLLM endpoint are kept separate from the normal CI suite. `test_vllm_integration.py` skips automatically when `VLLM_URL` is missing, points to localhost, or cannot be reached.

Run these tests manually against a deployed endpoint:

```
VLLM_URL=<live-endpoint> .venv/bin/python -m pytest \
 backend/tests/test_vllm_integration.py -v
```

The integration suite checks connectivity, confirms that the `inclusify` adapter is loaded, and tests English and Hebrew inference and severity mapping.

9.4 Frontend Tests

Frontend tests use Jest, Testing Library, and `jest-axe`. The Jest configuration uses the `jsdom` environment and loads shared setup from `jest.setup.ts`.

Tests are stored in two locations:

- `frontend/__tests__/` for page and feature-level tests.
- Co-located `*.test.tsx` files beside the component being tested.

For example, `PrivateModeToggle.test.tsx` checks rendering, interaction, and accessibility, while `SeverityBadge.test.tsx` checks severity variants, styling, and accessibility.

Run the frontend suite with:

```
cd frontend
```

```
npm test
```

9.5 Load Tests

Load tests are implemented with Locust under `backend/loadtests/`. They cover the two main user-facing bottlenecks:

- document upload through `/api/v1/ingestion/upload`;
- text analysis through `/api/v1/analysis/analyze`.

The available scripts support file uploads, mixed Hebrew and English analysis, and English-only analysis for isolating model performance. Real user documents must not be committed as fixtures.

10. CI/CD and Release Process

Inclusify uses GitHub Actions for continuous integration. The main workflow, `.github/workflows/ci.yml`, runs on every pull request and on pushes to `main`.

It performs the following checks:

- `ruff` linting for the backend.
- Backend tests with `pytest`, excluding live vLLM deployment and integration tests.
- Frontend linting with ESLint.
- Type checking with `tsc --noEmit`.
- Repository sanity checks, including failure if a `.env` file is committed.

10.1 Release Process

Frontend and backend releases are deployed automatically by Railway after changes are merged into `main`. Railway builds the corresponding Dockerfiles and deploys the updated services.

Model-serving changes are deployed separately through Modal:

```
modal deploy infra/modal/vllm_app.py
```

The vLLM integration workflow is also separate from normal CI. It runs nightly or manually against a live endpoint using `VLLM_URL` and `VLLM_API_KEY`.

10.2 Branch and Pull Request Conventions

Development should be performed on feature branches and merged into `main` through pull requests. The pull request template requires a description, change type, testing details, screenshots when relevant, and a final checklist.

Commit messages should follow:

```
type(scope): subject
```

For example:

```
fix(auth): handle expired JWT tokens
```

11. Conventions, Quirks, and Gotchas

These decisions may look unusual, but they are intentional and should not be changed without understanding their impact.

Quirk	Why it is intentional	What breaks if changed
CPU-only PyTorch	The backend installs CPU-only Torch because model inference runs remotely on Modal.	Installing GPU/CUDA packages unnecessarily increases the Docker image size without improving backend performance.
Docling warm-up and offline mode	The Docker build performs a real Docling conversion before enabling `HF_HUB_OFFLINE=1`, so the required model weights are baked into the image.	Removing the warm-up can cause the first production upload to fail because the container runs offline.
Docling subprocess recycling	Document parsing runs in recycled spawned subprocesses because running Docling inside the API process caused memory growth and OOM failures.	Moving parsing back in-process may reintroduce the same memory leak and out-of-memory problem.
`MODEL_SCALE_TO_ZERO=true`	The model server scales down when idle, and health checks are designed not to wake the GPU.	Polling the model directly or disabling scale-to-zero can keep the GPU active and create approximately `$430/month` in idle costs.
Level-split logging	`INFO` and `DEBUG` logs go to stdout, while `WARNING` and above go to stderr, because Railway determines severity from the output stream.	Replacing this with basic logging can cause normal information logs to appear as errors.
`HybridDetector` is a historical name	Detection is currently LLM-only, even though the class name suggests a hybrid system.	Treating the database rules and glossary as an active detection engine may lead developers to extend a pipeline that does not exist.
`NEXT_PUBLIC_API_URL` is build-time configuration	The backend URL is embedded into the frontend during the Next.js build.	Changing only the runtime environment will not update the existing frontend build; the frontend must be rebuilt.

Quirk	Why it is intentional	What breaks if changed
i18n discipline	All user-visible strings are stored in both `frontend/messages/en.json` and `frontend/messages/he.json`.	Hardcoded strings or missing translations can cause one locale, especially the Hebrew UI, to display fallback or incomplete content.
Vestigial code and schema conventions	Prisma packages remain in `frontend/package.json`, but they are unused, while `db/schema.sql` is the canonical database schema. The valid admin role is `site_admin`.	Using Prisma as the schema source or treating `org_admin` as a valid database role can create schema and authorization inconsistencies.

12. Extension Points and Future Work

Inclusify is designed to support several future extensions already identified in the project roadmap and original requirements.

1. **Audit Agent and RAG citations** — A future Audit Agent could evaluate findings against external guidelines and attach supporting citations to explanations. This would mainly extend the analysis pipeline in `backend/app/modules/analysis/` and the results UI in the frontend.
2. **Improved Hebrew and outdated-term detection** — Hebrew analysis and detection of outdated terminology can be improved through additional training data, prompt refinement, model updates, and targeted evaluation. This work would affect the datasets and evaluation assets under `data/` and `ml/`, as well as prompting and post-processing in `backend/app/modules/analysis/.`
3. **Glossary and rules integration** — The `glossary_terms` and `rules` tables already exist but are not currently used by the detection flow. They could be connected to the analysis service and exposed through an admin-editable glossary and rules interface. Alternatively, they should be removed if the project remains fully LLM-based.
4. **Sensitivity and category controls** — The original requirements include adjustable sensitivity thresholds and the ability to enable or disable rule categories. These controls would require admin settings in the frontend and configuration handling in the backend analysis pipeline.
5. **Additional file formats** — The ingestion pipeline can be expanded to accept more document types beyond the currently supported formats. This would require new validators and parsers in backend/app/modules/ingestion/ and corresponding updates to the upload interface.

These extensions should reuse the existing modular analysis, ingestion, database, and admin components rather than introducing separate parallel flows.